

// JavaJub

---

# Собес в Т1

## Иннотех

## Java Junior (Middle)

Вопросы, задачи и подготовка  
к live-coding и техническому интервью

---

Стек

Java • Spring Boot • Hibernate • PostgreSQL • Kafka • Docker • Microservices

[// содержание](#)

# Содержание

|           |                            |    |
|-----------|----------------------------|----|
| <b>01</b> | Про Т1 и формат собеседа   | 3  |
| <b>02</b> | Стек по вакансии           | 5  |
| <b>03</b> | Java Core                  | 6  |
| <b>04</b> | Многопоточность            | 8  |
| <b>05</b> | Коллекции                  | 10 |
| <b>06</b> | JVM, память и GC           | 11 |
| <b>07</b> | Spring и транзакции        | 12 |
| <b>08</b> | Hibernate и JPA            | 14 |
| <b>09</b> | PostgreSQL                 | 15 |
| <b>10</b> | Kafka                      | 17 |
| <b>11</b> | Микросервисы и архитектура | 18 |
| <b>12</b> | Code Review и практика     | 20 |
| <b>13</b> | Рет-проекты                | 22 |
| <b>14</b> | План подготовки + чек-лист | 23 |

# 01 Про Т1 и формат собеседа

ИТ-Холдинг Т1 — крупнейший частный ИТ-интегратор России. 30 000+ сотрудников, проекты для Сбера, ВТБ, Росатома, министерств. Иннотех — дочерняя компания Т1, разрабатывающая продукты для банков (кредиты физлиц, страхование).

Вакансия заявлена как «Junior», но требования чётко Middle: 3+ лет коммерческого опыта Java, 2–3 года на высоконагруженных проектах, микросервисы, Kafka. Готовиться нужно на Middle.

## ► Структура собеседования (~1.5–2 часа)

| Блок                            | Время   | Что проверяют                              |
|---------------------------------|---------|--------------------------------------------|
| Опыт + проект                   | ~10 мин | Стек, нагрузка, зона ответственности       |
| Java Core + многопоточность     | ~25 мин | equals/hashCode, HashMap, volatile, CAS    |
| Spring + Hibernate + транзакции | ~20 мин | Жизненный цикл, прокси, N+1, Propagation   |
| Kafka + БД + микросервисы       | ~20 мин | Гарантии, MVCC, Saga, Circuit Breaker      |
| Code Review                     | ~15 мин | Найти 5–10 проблем в Spring-сервисе        |
| SQL практика                    | ~15 мин | JOIN 3 таблиц + GROUP BY + HAVING          |
| Архитектурный кейс              | ~10 мин | Rate limiting, health checks, UUID vs Long |

### ФИШКА • Специфика Т1/Иннотех

На собеседах в Т1 любят: Code Review реального Spring-сервиса, архитектурные кейсы (rate limiting между сервисами с разной TPS, circuit breaker при «моргании сети»), SQL-серии из 4 задач. Сдвиг 2026: от синтаксиса к инженерному мышлению — обоснование выбора через производительность и стоимость.

### ВНИМАНИЕ • Заявлен Junior, но...

Требования: 3+ лет опыта Java, 2–3 года на высоконагруженных, микросервисы, Kafka/RabbitMQ, Docker, тесты. Это полноценный Middle. Готовьтесь соответственно.

## 02 Стек по вакансии

---

### ► Основной стек

- Java (3+ лет) — Core, многопоточность, Stream API
- Spring Boot — основной фреймворк, микросервисы
- Hibernate / JPA — ORM, маппинг, кэширование
- PostgreSQL — основная СУБД
- Kafka / RabbitMQ — брокеры сообщений
- Docker — контейнеризация
- JUnit, Mockito — тестирование
- Микросервисная архитектура — опыт 2–3 года
- Git, Maven/Gradle — сборка и CI/CD

### ► Будет плюсом

- Kubernetes / OpenShift — оркестрация
- Redis — кэширование
- Elasticsearch — полнотекстовый поиск
- gRPC — межсервисное взаимодействие
- Resilience4j — Circuit Breaker, Retry
- Опыт в банковских/страховых проектах

## 03 Java Core

- Расскажите про контракт equals/hashCode. Что случится с HashMap при константном hashCode?

→ Если `a.equals(b)`, то hashCode одинаков. Обратное необязательно. При константном hashCode — все в одном bucket: Java 7 — список  $O(n)$ , Java 8+ при 8 коллизиях — red-black tree  $O(\log n)$ . Деградация. При рандомном — объект «потеряется» при `get()`.

- Почему ключ HashMap должен быть иммутабельным?

→ Если изменить поле, участвующее в hashCode — объект окажется в «неправильном» bucket. `contains()` вернёт false. Объект потерян — ни найти, ни удалить. Утечка памяти. Решение: immutable-ключи (`String`, `Integer`, `record`).

- Как написать иммутабельный класс?

→ `final class` (нельзя наследовать). `private final` поля. Нет сеттеров. Конструктор с `defensive copy` для mutable-аргументов. Геттеры возвращают `defensive copy` для mutable-полей (`List` → `Collections.unmodifiableList()`). Пример: `record` в Java 16+ — `immutable` по умолчанию.

- String vs StringBuilder vs StringBuffer.

→ `String`: `immutable`, при конкатенации создаёт новый объект. `StringBuilder`: `mutable`, НЕ потокобезопасный (быстрый). `StringBuffer`: `mutable`, потокобезопасный (`synchronized` — медленнее). В цикле конкатенация `String` →  $O(n^2)$ , `StringBuilder` →  $O(n)$ . Компилятор оптимизирует простые случаи, но не циклы.

- Чем опасны типы-обёртки (Integer, Boolean)?

→ NPE при `unboxing null`: `Integer i = null; int x = i;` → NPE. Лишняя память: `Integer` = 16 байт vs `int` = 4 байта. Медленнее: автобоксинг создаёт объекты. `IntegerCache`: `valueOf(127) == valueOf(127)` → `true`, `valueOf(128)` — `false`. Мораль: для вычислений — примитивы, для коллекций — обёртки.

- Лямбда-выражения: почему переменные должны быть `effectively final`?

→ Лямбда захватывает копию значения переменной, не ссылку на неё. Если переменная изменится после захвата — копия устареет → неочевидное поведение. Компилятор требует `final` или `effectively final` для защиты от ошибок. Workaround: `AtomicInteger` или массив из одного элемента (антипаттерн).

- Stream API: что произойдёт при двух терминальных операциях?

→ `IllegalStateException`: `stream has already been operated upon or closed`. Стрим одноразовый — после терминальной операции закрывается. Если нужно два результата — `Collectors.teeing()` (Java 12+) или два разных стрима из одного источника.

- Parallel streams: в чём опасность?

→ Все `parallel streams` делят общий `ForkJoinPool.commonPool()`. Если один параллельный стрим заблокировался на I/O — остальные ждут. Решение: кастомный `ForkJoinPool`: `new ForkJoinPool(4).submit(() -> stream.parallel()...)`. Не использовать для I/O-bound задач.

- Java 17/21: что нового для повседневной разработки?

→ `Records`: `immutable data`-класс с `equals/hashCode/toString`. `Sealed Classes`: ограничение иерархии (`exhaustive switch`). `Pattern Matching`: `instanceof` без `cast`, `switch` по `enum`. `Virtual Threads`: миллионы лёгких потоков (I/O-bound). `Sequenced Collections`: `addFirst()`, `getLast()`, `reversed()`. `Structured Concurrency`: `StructuredTaskScope` для управления задачами.

## 04 Многопоточность

Многопоточность — явный фокус вакансии (высоконагруженные проекты). Спрашивают глубоко: volatile, CAS, happens-before, пулы, Virtual Threads.

- volatile: что гарантирует и почему недостаточно для i++?
  - Видимость (запрет кэширования), happens-before (volatile write → read), запрет reordering. i++ = read+increment+write — три операции. Между ними другой поток может прочитать старое значение. Решения: AtomicInteger (CAS), synchronized, LongAdder.
- synchronized на двух методах одного объекта — будут ли работать параллельно?
  - Нет. Оба используют один монитор — this. Пока один поток в methodA(), другой не может войти в methodB(). Решение: synchronized(privateLockA) и synchronized(privateLockB) — разные мониторы.
- Чем synchronized(this) отличается от synchronized(privateLock)?
  - this доступен извне — любой может случайно или намеренно synchronized(obj) на том же объекте. privateLock (private final Object lock = new Object()) — полный контроль, никто извне не может захватить.
- happens-before: какие пары существуют?
  - volatile write → read. synchronized release → acquire. Thread.start() → первая инструкция потока. Последняя инструкция → Thread.join(). final поля после конструктора (если this не утёк). Запись в ConcurrentHashMap → чтение. Без happens-before видимость НЕ гарантирована.
- Race condition: как воспроизвести и как бороться?
  - Check-then-act без атомарности: if (map.containsKey(k)) map.get(k). Между проверкой и получением другой поток может удалить. Решение: computeIfAbsent (атомарно). Воспроизвести: Thread.sleep(1) между check и act + два потока. Бороться: атомарные операции, synchronized, CAS.
- Virtual Threads (Java 21): pinning problem.
  - Virtual Thread на synchronized блоке → pinning (привязка к platform thread, теряется лёгкость). Причина: synchronized работает через монитор объекта, JVM не может «припарковать» виртуальный поток. Решение: ReentrantLock вместо synchronized. Virtual Threads идеальны для I/O-bound, НЕ для CPU-bound (нет preemption).
- Structured Concurrency — что это?
  - StructuredTaskScope (preview в Java 21): управление временем жизни задач. Все подзадачи привязаны к scope — при закрытии scope все незавершённые отменяются. ShutdownOnFailure: при первой ошибке все остальные отменяются. ShutdownOnSuccess: при первом успехе остальные отменяются. Замена разрозненным CompletableFuture.
- Иммутабельность как способ потокобезопасности.
  - Immutable-объекты потокобезопасны по определению — нет shared mutable state. Нет необходимости в synchronized/volatile. Примеры: String, Integer, LocalDate, record. В многопоточном коде: вместо изменения объекта — создание нового. CopyOnWriteArrayList — тоже вариант (иммутабельность массива при записи).

## 05 Коллекции

---

- **HashMap: расчёт бакета на примере.**
  - Формула:  $(n-1) \& \text{hash}(\text{key})$ . Если  $\text{capacity}=16$  ( $n=16$ ):  $(15) \& \text{hash}$ . 15 в бинарном = 1111.  $\text{hash} = 2\_000\_000\_000$ : 2000000000 & 15 = 0 (последние 4 бита = 0000). Результат: bucket 0. Поэтому  $\text{capacity}$  — степень двойки, и  $\text{hash}$  дополнительно перемешивается (XOR верхних бит с нижними).
- **HashMap vs Hashtable vs LinkedHashMap vs WeakHashMap.**
  - **HashMap**: не потокобезопасен, null-ключ. **Hashtable**: потокобезопасен (synchronized на всё), устаревший, нет null. **LinkedHashMap**: порядок вставки (или access order для LRU). **WeakHashMap**: ключи через WeakReference — GC собирает неиспользуемые записи (для кэшей). **ConcurrentHashMap** — замена Hashtable.
- **Два одинарных индекса vs один составной — что лучше?**
  - Зависит от запроса. WHERE  $a=1$  AND  $b=2$ : составной  $(a,b)$  — один Index Scan. Два одинарных: Bitmap Index Scan + Bitmap AND — медленнее. WHERE  $a=1$  OR  $b=2$ : два одинарных — Bitmap OR. Составной  $(a,b)$  не поможет для WHERE  $b=2$  (leftmost prefix). Общее правило: составной для AND-запросов, одинарные для OR или независимых WHERE.
- **ConcurrentModificationException: причина и решение.**
  - Модификация коллекции во время итерации не через итератор. modCount-счётчик: при следующем next() проверяется. Решения: Iterator.remove(), CopyOnWriteArrayList, ConcurrentHashMap, removeIf() (Java 8+), stream + filter + collect в новый список.
- **CopyOnWriteArrayList: плюсы и минусы.**
  - Плюсы: lock-free чтение (читатели не блокируются), fail-safe итератор (работает со snapshot). Минусы: при каждой записи — копирование всего массива → дорого. Подходит: много чтений, редкие записи (listeners, подписчики, конфигурации). Не подходит: частые записи.

## 06 JVM, память и GC

---

- Структура памяти JVM: где что хранится?

→ *Heap*: объекты (*Young=Eden+Survivor, Old*). *Stack*: фреймы методов (на каждый поток свой, LIFO), примитивы, ссылки. *Metaspace*: метаданные классов, статические поля. *Code Cache*: JIT-скомпилированный код. *Direct Memory*: NIO-буферы. *StackOverflowError*: переполнение стека (рекурсия). *OOM*: переполнение *heap/metaspace/direct*.

- Generational ZGC — что нового?

→ ZGC с Java 21 стал *generational* (раздельная сборка *Young/Old*). Паузы  $<1ms$  даже на терабайтных *heap*. Предыдущий ZGC был *non-generational* (собирал весь *heap*). *Generational ZGC* — стандарт для *high-load* в 2026. *G1* — для случаев, когда ZGC *overkill* (*heap*  $<4\text{ GB}$ ).

- Как воспроизвести OOM?

→ *static Map* без *eviction*: статический *Map*, добавляешь объекты в цикле → *heap exhaustion*. Бесконечное создание *Thread*: каждый поток =  $\sim 1\text{MB}$  стека → *native OOM*. *Metaspace*: динамическая генерация классов (CGLIB, ASM). *Direct Memory*: *ByteBuffer.allocateDirect()* без освобождения.

- Виды ссылок: Strong, Soft, Weak, Phantom.

→ *Strong*: обычная, GC не трогает. *Soft*: GC собирает при нехватке памяти (кэши). *Weak*: GC собирает при первой сборке (*WeakHashMap*). *Phantom*: *get()* всегда *null*, уведомление через *ReferenceQueue* (постфинализационная очистка). Циклические ссылки: GC соберёт — определяет достижимость от GC Roots, а не *reference counting*.



## 07 Spring и транзакции

- Singleton-бин: потокобезопасен ли?
  - НЕТ. Singleton означает один экземпляр на контекст, но это НЕ делает его потокобезопасным. Если есть mutable state (поля, не final) — race condition. Решения: stateless бины (без полей-состояний), ThreadLocal, synchronized, ConcurrentHashMap. В Spring MVC: контроллеры и сервисы — singleton, но stateless.
- @Transactional на private методе — что произойдёт?
  - Ничего — транзакция не создастся. CGLIB-прокси наследует класс, но private методы не переопределяемы → прокси их не видит. JDK Dynamic Proxy работает через интерфейс — private вообще не в интерфейсе. Решение: сделать метод package-private или public.
- Два @Transactional метода в одном классе: A() вызывает B(). Сколько транзакций?
  - Одна. Вызов this.B() минуем прокси → @Transactional на B() не работает. B() выполняется в транзакции A() (Propagation.REQUIRED по умолчанию). Если нужна отдельная транзакция для B() — вынести в другой бин или self-injection через @Lazy.
- Когда нужен REQUIRES\_NEW?
  - Когда нужна НЕЗАВИСИМАЯ транзакция, которая сохранится даже при rollback основной. Примеры: запись аудит-лога (должна остаться при ошибке), отправка уведомления, счётчик попыток. Минус: две открытые транзакции одновременно → два соединения из пула.
- @Transactional + долгий REST-вызов внутри — чем опасно?
  - Транзакция удерживает соединение к БД всё время REST-вызова (секунды). Пул соединений исчерпается → все потоки ждут → сервис висит (connection starvation). Решение: read → close transaction → REST → open transaction → write. Или вынести REST-вызов за @Transactional.
- В какой момент происходит коммит @Transactional?
  - После успешного завершения метода (без исключений). Прокси: try { beginTransaction; target.method(); commit; } catch { rollback; }. Rollback по умолчанию: RuntimeException и Error. Checked — НЕ откатывают (нужен rollbackFor). flush != commit: flush отправляет SQL, commit фиксирует.
- AOP: кейс «замерить время всех методов сервиса».
  - @Aspect + @Around("execution(\* com.example.service.\*(..))"). ProceedingJoinPoint: long start = System.nanoTime(); Object result = pjp.proceed(); long elapsed = System.nanoTime() - start; log.info("{} took {} ms", methodName, elapsed/1\_000\_000); return result. В проде: Micrometer @Timed вместо ручного AOP.
- Конфликт зависимостей: как искать?
  - mvn dependency:tree — показывает дерево зависимостей. Ищем разные версии одной библиотеки. Gradle: ./gradlew dependencies. Решение: exclusion в Maven/Gradle, dependencyManagement/BOM, force в Gradle. Классика: Jackson 2.14 из Spring vs Jackson 2.12 из другой библиотеки.

## 08 Hibernate и JPA

---

- JPA vs Hibernate.

→ JPA — спецификация (набор интерфейсов и аннотаций). Hibernate — реализация (конкретная библиотека). EntityManager — JPA-интерфейс, Session — Hibernate-интерфейс. Можно сменить Hibernate на EclipseLink без изменения кода (если использовать только JPA API).

- Каскадные операции: что делает CascadeType.ALL?

→ PERSIST + MERGE + REMOVE + REFRESH + DETACH. Опасность: CascadeType.REMOVE при OneToMany — удаление родителя удалит ВСЕХ детей. Если детей 10 000 — 10 000 DELETE запросов. orphanRemoval=true — удаляет «осиротевших» детей при удалении из коллекции.

- Entity как ключ HashMap — что может пойти не так?

→ Если equals/hashCode используют lazy-связи (ManyToOne, OneToMany) — при первом вызове hashCode загрузится N связанных сущностей (N+1). Хуже: если связи двусторонние — StackOverflowError (бесконечная рекурсия). Решение: equals/hashCode только по @Id (или бизнес-ключу без связей). Lombok @EqualsAndHashCode(onlyExplicitlyIncluded=true).

- JdbcTemplate — когда уместен вместо Hibernate?

→ Сложные аналитические запросы (оконные функции, CTE). Batch-операции (тысячи INSERT). Запросы, где ORM генерирует неоптимальный SQL. Миграция legacy-кода. Простые CRUD без сложных связей. В Т1: JdbcTemplate используют для ETL и репортинга, Hibernate — для бизнес-логики.

## 09 PostgreSQL

PostgreSQL — главная БД вакансии. В Т1/Иннотех SQL-практика обязательна: серия из 4 задач на одном собеседе. Спрашивают MVCC глубоко, VACUUM, индексы.

- В PostgreSQL нет Read Uncommitted — почему?
  - Минимальный уровень — Read Committed. PostgreSQL использует MVCC: каждая транзакция видит snapshot, грязные данные физически невозможно прочитать (нет dirty read). Read Uncommitted в PG ведёт себя как Read Committed. Любят спрашивать — это особенность PG.
- Repeatable Read в PG решает фантомное чтение — почему?
  - В стандарте SQL Repeatable Read допускает phantom reads. В PG благодаря MVCC (snapshot isolation): транзакция видит snapshot данных на момент начала. Новые строки, вставленные другими транзакциями, не видны → нет фантомов. Но: при попытке UPDATE конфликтующей строки — serialization failure → retry.
- UUID vs Long как PK: что выбрать?
  - Long: компактный (8 байт), быстрый B-tree (последовательная вставка), auto-increment. UUID: скрывает количество записей (безопасность), не нужен центральный генератор (распределённые системы). Минусы UUID: 16 байт, случайная вставка в B-tree (page split, fragmentation). Решение: UUIDv7 (time-ordered) — сохраняет преимущества UUID без проблем B-tree.
- Кейс: «навесили индексов на все поля — запись стала медленнее».
  - Каждый INSERT/UPDATE/DELETE обновляет BCE индексы. 10 индексов = 10x overhead на запись. VACUUM тоже замедляется. REINDEX может понадобиться. Решение: индексы только по паттернам запросов (WHERE, JOIN, ORDER BY). Мониторинг: pg\_stat\_user\_indexes → проверить idx\_scan (сколько раз использовался).
- pg\_stat\_activity и pg\_stat\_statements — что показывают?
  - pg\_stat\_activity: текущие запросы, их состояние (active/idle/waiting), PID, query\_start. Для поиска long-running queries и блокировок. pg\_stat\_statements: статистика по запросам (calls, total\_time, mean\_time, rows). Для поиска самых тяжёлых запросов. Нужен extension (CREATE EXTENSION pg\_stat\_statements).

### ► SQL-задачи из собеседов Т1

#### TOP-5 платежей каждого клиента

```
SELECT * FROM (
  SELECT c.name, p.amount, p.created_at,
    ROW_NUMBER() OVER (
      PARTITION BY c.id ORDER BY p.amount DESC
    ) AS rn
  FROM clients c
  JOIN payments p ON p.client_id = c.id
) t WHERE rn <= 5;
```

#### Найти дубли по email

```
SELECT email, COUNT(*) AS cnt
FROM users
GROUP BY email
HAVING COUNT(*) > 1
ORDER BY cnt DESC;
```

## 10 Kafka

---

- Kafka vs RabbitMQ: ключевые различия.

→ Kafka: pull (consumer сам забирает), хранит сообщения (retention), log compaction, высокая пропускная способность, горизонтальное масштабирование. RabbitMQ: push (брокер отправляет), удаляет после ACK, rich routing (exchanges), проще для простых сценариев. В Т1: Kafka для event-driven, RabbitMQ для task queues.

- ZooKeeper vs KRaft — что изменилось?

→ Старые версии Kafka: ZooKeeper хранил метаданные (контроллер, брокеры, топика). KRaft (Kafka Raft, с Kafka 3.3+): метаданные хранятся в самом Kafka через Raft-консенсус. Проще деплой (нет отдельного ZooKeeper-кластера), быстрее восстановление, меньше точек отказа. ZooKeeper deprecated с Kafka 4.0.

- Spring Kafka: основные компоненты.

→ KafkaTemplate: отправка сообщений (send). @KafkaListener: получение (метод с @KafkaListener(topics="...")). ConcurrentKafkaListenerContainerFactory: настройка concurrency, error handler, retry. ConsumerConfig: bootstrap-servers, group-id, deserializer. ProducerConfig: acks, retries, serializer.

- Kafka Streams vs обычный Consumer.

→ Consumer: ты сам управляешь offset, обработкой, сохранением. Kafka Streams: декларативная обработка потоков (filter, map, groupBy, aggregate, join). Stateful (RocksDB для локального состояния). Exactly-once semantics встроены. Для простых задач — Consumer, для stream processing (агрегация, join потоков) — Kafka Streams.

- Как обеспечить сохранность данных в Kafka?

→ replication.factor=3 (каждая партиция на 3 брокерах). min.insync.replicas=2 (минимум 2 реплики подтверждают). acks=all (producer ждёт подтверждения от всех ISR). unclean.leader.election.enable=false (не выбирать лидера из отставших реплик). Если 1 брокер упал — данные не потеряны.

# 11 Микросервисы и архитектура

- Кейс: сервис А — 1000 TPS, сервис В — 200 TPS. Как защитить В?
  - *Rate limiter* на стороне А (ограничить исходящие). *Kafka* как буфер: А пишет в топик → В читает со своей скоростью (*backpressure*). *Circuit Breaker*: при перегрузке В → А получает *fallback*. *Bulkhead*: отдельный пул потоков для вызовов В (не блокирует остальные). Самое надёжное: *Kafka-буфер + rate limiter*.
- Кейс: сеть «моргнула» на 2 секунды. Что делать?
  - *Retry* с *exponential backoff + jitter* (не все клиенты ретраят одновременно). *Circuit Breaker*: после *N* ошибок → *OPEN* (не шлём запросы) → *HALF\_OPEN* (тестовый) → *CLOSED*. *Idempotency-Key*: чтобы *retry* не создал дубль. *Timeout*: не ждать бесконечно (5–10 сек). *Fallback*: кэшированный ответ или *graceful degradation*.
- Кейс: два дата-центра, один упал. Как балансировщик узнает?
  - *Health checks*: *HTTP GET /health* каждые *N* секунд. Если 3 подряд неуспешных → убрать из ротации. *Service Discovery* (*Eureka/Consul*): сервис регистрируется, посылает *heartbeat*. Если *heartbeat* пропал → *deregister*. *DNS failover*: *TTL 30* сек, при смене А-записи трафик переключается. *Readiness probe* в *K8s*: *Pod* не *ready* → не получает трафик.
- *UUID* vs *Long* как *PK* для соцсети на 2 млн пользователей.
  - *Long*: компактный, быстрый, но раскрывает количество (*user/12345* → 12345 пользователей). *UUID*: безопаснее, но 2х размер индекса. *UUIDv7* (*time-ordered*): лучший выбор — безопасность *UUID* + последовательность для *B-tree*. Для 2 млн: *Long* достаточен, но для безопасности *API* — *UUID* в *URL*, *Long* внутри.
- *Circuit Breaker*: три состояния.
  - *CLOSED*: всё работает, запросы проходят. При *N* ошибок (или % ошибок > *threshold*) → *OPEN*. *OPEN*: все запросы блокируются, возвращается *fallback*. Через *waitDuration* → *HALF\_OPEN*. *HALF\_OPEN*: пропускает *N* тестовых запросов. Если успешны → *CLOSED*. Если неуспешны → *OPEN*. *Resilience4j*: `@CircuitBreaker(name="backend", fallbackMethod="fallback")`.

## 12 Code Review и практика

В T1/Иннотех Code Review — обязательная часть собеседа (~15 мин). Дают 50–200 строк Spring-сервиса. Нужно найти 5–10 проблем. Вот типичные:

### ► Типичные проблемы в Code Review

- Field injection (@Autowired на поле) → constructor injection + final
- System.out.println → SLF4J логгер (log.info/warn/error)
- double для денег → BigDecimal (потеря точности:  $0.1 + 0.2 \neq 0.3$ )
- optional.isPresent() + get() → map().orElseGet()
- findAll().stream().filter().count() == 1 → findById или existsById
- REST-клиент через new → бин с пулом соединений
- @Transactional без rollbackFor для checked-исключений
- @Transactional + долгий REST-вызов внутри (connection starvation)
- Кэш в HashMap без eviction → утечка памяти → Caffeine с maxSize
- Endpoint без пагинации → Pageable
- Mutable shared state без синхронизации → потокобезопасная альтернатива
- Конкатенация String в цикле → StringBuilder
- Нарушение SRP → вынести стратегию через интерфейс

### ► Задача: найти проблемы

```
@Service
public class PaymentService {
    @Autowired private PaymentRepo repo; // ← field injection
    private Map<Long, Payment> cache = new HashMap<>(); // ← no eviction

    @Transactional
    public Payment process(Long id) {
        if (cache.containsKey(id)) return cache.get(id); // ← not atomic
        Payment p = repo.findById(id).get(); // ← NoSuchElementException
        double total = p.getAmount() * 1.2; // ← double для денег
        System.out.println("Total: " + total); // ← sout
        String result = callExternalApi(p); // ← долгий REST в @Transactional
        cache.put(id, p); // ← HashMap не потокобезопасен
        return p;
    }
}
```

8 проблем: 1) field injection, 2) HashMap без eviction, 3) containsKey+get не атомарно, 4) .get() без isPresent, 5) double для денег, 6) sout, 7) REST в @Transactional, 8) HashMap не потокобезопасен.

## 13 Pet-проекты

Pet-проект покажет, что вы умеете не только отвечать на вопросы. Для Т1 — банковские/страховые проекты в плюс.

### ► Проект 1. Платёжный сервис с идемпотентностью

- Spring Boot 3 + PostgreSQL + Kafka + Redis
- Idempotency-Key: дедупликация платежей через Redis SET NX
- Transactional Outbox: событие в outbox-таблицу + Debezium/poller → Kafka
- Circuit Breaker (Resilience4j) для внешних API
- Docker Compose + Testcontainers + JUnit 5

### ► Проект 2. Кредитный конвейер (мини)

- Spring Boot + PostgreSQL + Kafka
- Saga (orchestration): заявка → скоринг → одобрение → выдача
- Компенсирующие транзакции при отказе
- @Version для оптимистичной блокировки
- Мониторинг: Micrometer + Prometheus + Grafana

### ► Проект 3. Event-driven уведомления

- Spring Boot + Kafka + PostgreSQL
- Consumer с идемпотентной обработкой (processed\_events)
- DLQ для failed messages + retry topic
- Rate limiter для внешнего SMS-шлюза
- Structured logging с traceId через MDC

#### СОВЕТ • Для Т1

На собеседе спросят: что было сложно, почему такой стек, что бы переделал. Для Т1 идеально: показать опыт с транзакциями, Kafka, идемпотентностью — это их ежедневные задачи в банковских проектах.

## 14 План подготовки + чек-лист

### ► За 2–3 недели

- Повторить Java Core: equals/hashCode, String, generics, Stream API, Java 21
- Многопоточность: volatile, CAS, happens-before, ThreadPoolExecutor, Virtual Threads
- Spring: жизненный цикл бина, @Transactional прокси, Propagation, AOP
- PostgreSQL: MVCC, VACUUM, EXPLAIN ANALYZE, покрывающие индексы, UUID vs Long
- Kafka: Consumer Group, ordering, Outbox, idempotency, KRaft
- 20–30 задач LeetCode Easy+Medium: HashMap, two pointers, binary search
- Pet-проект: Spring Boot + Kafka + PostgreSQL + Docker Compose

### ► За неделю

- 2–3 мок-интервью
- Все вопросы из гайда ВСЛУХ
- Code Review: потренироваться находить проблемы в Spring-коде
- SQL: JOIN 3 таблиц + GROUP BY + HAVING + оконные функции
- Архитектурные кейсы: rate limiting, circuit breaker, health checks

### ► В день собеседа

- Камера, микрофон, интернет — за 30 минут
- Рассуждать вслух — молчание хуже «дай подумать»
- Code Review: начинай с самых очевидных проблем (sout, field injection)
- SQL: сначала напиши SELECT FROM JOIN, потом GROUP BY HAVING
- 2–3 вопроса в конце: про проект, команду, стек, процессы

#### ВНИМАНИЕ • Главное для Т1

В Т1/Иннотех Code Review и SQL — обязательные секции. Потренируйтесь находить 5–10 проблем в Spring-сервисе за 15 минут. SQL-серия из 4 задач: JOIN + GROUP BY + HAVING, TOP-N через ROW\_NUMBER, дубли, вторая зарплата.

### ► Финальный чек-лист

| Блок            | Готов, если можешь...                                                                |
|-----------------|--------------------------------------------------------------------------------------|
| Java Core       | Иммутабельный класс + effectively final + parallel streams опасность                 |
| Многопоточность | synchronized(this) vs privateLock + Virtual Threads pinning + Structured Concurrency |
| Коллекции       | HashMap расчёт бакета + WeakHashMap + CopyOnWriteArrayList                           |
| JVM/GC          | Generational ZGC + OOM диагностика + ссылочные типы                                  |



|              |                                                                               |
|--------------|-------------------------------------------------------------------------------|
| Spring       | Singleton НЕ потокобезопасен + @Transactional private + AOP замер времени     |
| Транзакции   | REQUIRES_NEW для аудита + self-call + connection starvation                   |
| Hibernate    | CascadeType.ALL опасность + entity как ключ<br>HashMap + JdbcTemplate когда   |
| PostgreSQL   | PG нет Read Uncommitted + MVCC phantom + UUID<br>vs Long + pg_stat_statements |
| Kafka        | KRaft vs ZooKeeper + Spring Kafka компоненты +<br>Kafka Streams vs Consumer   |
| Микросервисы | Rate limiting кейс + Circuit Breaker 3 состояния +<br>Saga orchestration      |
| Code Review  | Найти 8 проблем в Spring-сервисе за 15 мин                                    |
| SQL          | 4 задачи: JOIN+GROUP BY, TOP-N, дубли, вторая<br>зарплата                     |

---

## Удачи на собесе!

// git push origin offer

JavaJub